

Chapter 1 Introduction

1.1 Background and Functional Split Evolution

In the era of 5G and the Open Radio Access Network (O-RAN), the functional split of network architecture has become the key to balancing transport bandwidth requirements and deployment flexibility. Currently, the industry recognizes three mainstream functional split options: **Option 2 (F1)**, **Option 6 (nFAPI)**, and **Option 7.2x (Open Fronthaul)** [?]. Although these three options all aim to achieve hardware disaggregation, their underlying synchronization requirements and timing window maintenance logic differ significantly:



- **Split Option 2 (3GPP F1):** The split point is located between the PDCP and RLC layers [?]. Because it primarily processes non-real-time messages, its synchronization requirement is at the **millisecond (ms) level**. The synchronization mechanism mainly relies on the **Timing Synchronisation Status (TSS)** procedure defined by 3GPP for status reporting. Its timing maintenance is relatively relaxed; when packet delays occur, it primarily relies on buffer management and Radio Resource Control (RRC) re-establishment mechanisms, rarely involving precise physical-layer timing alignment.
- **Split Option 6 (SCF nFAPI):** The split point is located between the MAC and PHY layers [?]. Its synchronization requirement is at the **slot-level (sub-ms)**, primarily calculated through the **nFAPI P7 Node Sync** message exchange (based on t_1 to t_4 timestamps)

to determine the round-trip delay (RTD). When packets exceed the timing window (Out-of-Window), nFAPI allows the Virtualized Network Function (VNF) to actively adjust the **Slot Offset** of the physical node, adapting to network jitter by “moving the window,” which provides strong resilience.

- **Split Option 7.2x (O-RAN WG4):** The split point is located within the physical layer (High-PHY/Low-PHY) [?]. This demands **nanosecond (ns) level** precise phase synchronization and must rely on hardware-level **IEEE 1588 PTP** and **SyncE** [?]. Its timing windows (e.g., T_{2a} and T_{a4}) are extremely strict. If a packet experiences even slight displacement or delay, the Radio Unit (RU) typically **drops** the packet directly, which immediately degrades the Error Vector Magnitude (EVM) and signal quality.



1.2 Comparative Analysis of Timing Maintenance

Table 1.1 summarizes the technical characteristics of the three split architectures regarding synchronization maintenance.

Table 1.1: Comparison of Functional Splits and Timing Maintenance

Split Option	Core Interface	Sync Level	Sync Mechanism	Out-of-
Option 2	F1-C/U	Millisecond	Message-based (TSS/GTP-U)	Buffer n
Option 6	nFAPI	Slot-level	Message-based (P7 Node Sync)	Algorit
Option 7.2x	O-FH	Nanosecond	Hardware-based (PTP/SyncE)	Direct l

This architectural difference means that when facing non-ideal fron-

thaul networks, Option 6 exhibits a better delay tolerance compared to Option 7.2x, while providing more granular physical-layer control compared to Option 2.

1.3 Motivation and Problem Statement

The necessity of investigating high latency in Lower Layer Split (LLS) architectures is highlighted in the O-RAN Fronthaul working group specification [?]. Specifically, Annex L of the specification discusses considerations for operating in non-ideal transport environments. Long latency fronthaul links may break traditional Hybrid ARQ (HARQ) timelines, leaving only a tight 4 ms processing budget for the base station. This constraint often necessitates the use of real-time operating systems (RTOS) and dedicated hardware. However, for low- to medium-impact deployment scenarios, such as indoor small cells and fixed wireless access, the performance impact of delayed HARQ or predictive HARQ is negligible [?]. Thus, enforcing sub-millisecond latency constraints restricts the exploration of innovative, cost-effective deployments.

In this context, the nFAPI (Split Option 6) architecture inherently aligns with the requirements of non-ideal fronthaul deployments. By separating the MAC and PHY layers, Split 6 exhibits a much higher latency tolerance (typically 2 ms to 10 ms) compared to Split 7.2x. This enhanced tolerance allows the Virtualized Network Function (VNF) hosting the MAC layer to be deployed on General Purpose Processors (GPPs) in cloud environments, eliminating the strict requirement for expensive dedicated fiber and precise synchronization hardware. Furthermore, nFAPI incorporates timing proce-

dures that enable the MAC layer to adjust its scheduling behavior based on fronthaul latency conditions, directly addressing the issue of broken HARQ windows.

The rationale for migrating the MAC layer to a centralized cloud environment centers on the concept of resource pooling and elasticity. In 5G vRAN architectures, the computational load of the physical layer (L1) is primarily determined by relatively static parameters, such as system bandwidth and antenna configuration. In contrast, the processing complexity of the Layer 2 (L2) stack, particularly the MAC scheduler, exhibits a linear or even quadratic dependency on the number of active User Equipments (UEs) and traffic volume [?,?]. For instance, as the number of connected UEs increases, the MAC scheduler must continuously evaluate proportional fair metrics, process buffer status reports, and handle logical channels, leading to a substantial surge in CPU cycles [?]. By decoupling the MAC layer into a VNF, the cloud infrastructure can dynamically allocate virtual cores to accommodate fluctuating UE densities, maintaining cost-effective scaling compared to local PNF deployments.

However, this resource scaling introduces a new challenge for synchronization. As the number of UEs increases, the MAC processing time ($T_{\text{process_VNF}}$) inevitably lengthens within a given Transmission Time Interval (TTI). This extended processing duration compresses the remaining time margin available for fronthaul transmission latency. If the scheduling mechanism does not account for this endogenously increasing processing delay, static timing configurations will result in scheduling messages arriving at the PNF too late. Although Option 6 possesses flexible timing adjustment capabilities, its dynamic adjustment algorithms remain imma-

ture in existing open-source implementations, such as OpenAirInterface (OAI). Consequently, when confronted with non-deterministic operating system scheduling, UE-dependent processing load, and network congestion, the system frequently encounters “Missing Subframe” errors. This thesis selects Split 6 as the primary research subject, proposing an Adaptive Advance-Time Control architecture to dynamically compensate for both processing and transport latency variations, effectively overcoming these synchronization failures. A detailed analysis of this non-deterministic latency and the corresponding verification of these problems will be presented in Chapter ??, specifically isolating these issues in Scenario I (??).

1.4 Related Works



The evolution of Radio Access Network (RAN) architectures from Distributed RAN (D-RAN) to Open RAN (O-RAN) requires a thorough study of functional splits [?, ?, ?]. The MAC-PHY split (Option 6), standardized by the Small Cell Forum (SCF) as nFAPI [?, ?], is suitable for small cell deployments. It allows the MAC layer to be virtualized on commercial servers while the PHY layer remains on dedicated hardware.

The performance of the nFAPI split depends on the transport network. Practical deployments often use non-ideal fronthaul, such as Shared Ethernet or DOCSIS. Mehran and MacKenzie [?] demonstrated that uplink throughput drops to zero with a small increase in fronthaul delay. Furthermore, server processing delay is unpredictable. Khalaf *et al.* [?] indicated that server delay increases under high CPU utilization, causing the MAC scheduler to miss its timing window.

To address fronthaul latency, Huang and Zhang [?] proposed a distributed MAC scheduling scheme. While this scheme reduces the impact of delayed grants, it introduces a “split-brain” issue with multiple schedulers. Akgun *et al.* [?] utilized intelligent scheduling to adapt modulation based on interference, but did not solve the sub-millisecond timing of message delivery itself. Existing OpenAirInterface (OAI) reference models typically rely on static timing windows, leading to frequent “missing subframe” errors when network jitter exceeds configured limits. This paper bridges this gap by proposing a centralized Adaptive Advance-Time Control algorithm to dynamically manage scheduling timing.

1.5 Contributions



- **Delay Management in nFAPI Split 6:** We analyze and quantify the non-deterministic latency factors, including network jitter and server processing time, that lead to synchronization failures in disaggregated RAN deployments.
- **Adaptive Advance-Time Control Algorithm:** We propose a dynamic timing adjustment algorithm using a dual-loop architecture. The Convergence Optimization Loop estimates end-to-end jitter, while the VNF Timing Actuator adjusts scheduling lead times to ensure punctual delivery of P7 messages.
- **Experimental Validation:** We prototype the proposed mechanism on the OAI platform. Experimental results show that our algorithm eliminates “missing subframe” errors and maintains stable throughput under non-ideal fronthaul environments.

The rest of the thesis is organized as follows. Chapter ?? defines the system model/architecture considered in this paper. The proposed analytical model/method is given in Chapter ?. Chapter ? shows the numerical results. Concluding remarks are given in Chapter ?.

